

M Ű E G Y E T E M 1 7 8 2

BUDAPESTI MŰSZAKI ÉS GAZDASÁGTUDOMÁNYI EGYETEM
VILLAMOSMÉRNÖKI ÉS INFORMATIKAI KAR
MESTERSÉGES INTELLIGENCIA ÉS RENDSZERTERVEZÉS TANSZÉK

Elosztott adatgyűjtő rendszerek

Intelligens beágyazott rendszerek laboratórium

(BMEVIMIMA21)

Készítette:

Pál Patrik, Gavallér Dávid

AIQTQW, ES02RO

BUDAPEST, 2026. október 17.

Tartalomjegyzék

1. Bevezetés **2**

2. Mérési feladatok **3**

1. feladat 3
1.1. feladat 3
1.2. feladat 3
1.3. feladat 4
1.4. feladat 8
1.5. feladat 9
1.6. feladat 10
1.7. feladat 12
2. feladat 15
2.1. feladat 15
2.2. feladat 16
2.3. feladat 16
2.4. feladat 18
2.5. feladat 19
2.6. feladat 19

1. Bevezetés

A mérés célja

A mérés célja az elosztott rendszerű hálózatok és azok problémáinak megismerése gyakorlati példákon keresztül.

Az első fő feladatrészben két mitmótot felhasználva vizsgáljuk a szenzor és a gateway által rögzített jelek szinkronizációs problémáit. A feladat során a két mérőegység által rögzített jelek időbeli összehangolását végezzük el időbélyeg-transzformáció segítségével, majd a gateway mintavételi időpontjaiban interpolációval becsüljük a szenzorértékeket.

A méréshez használt eszközök

- Mitmótok
- Analóg interface kártya
- Oszcilloszkóp
- Számítógép
- Függvénygenerátor

2. Mérési feladatok

1. feladat

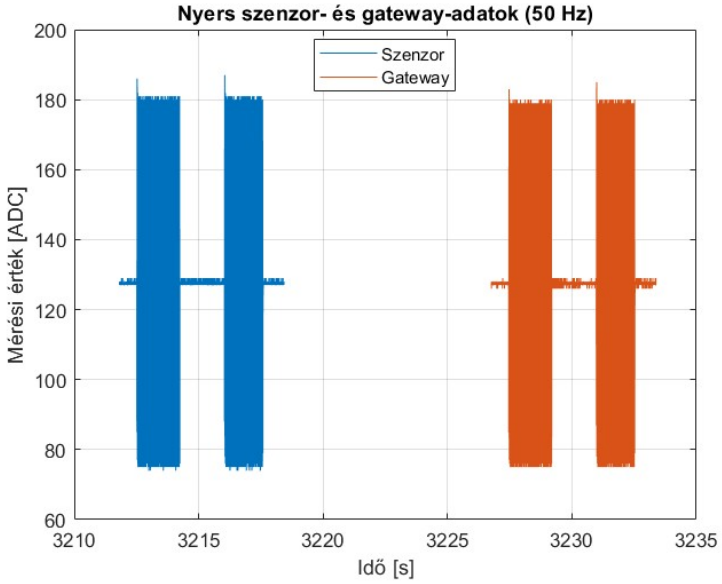
Az első feladat célja a különálló mérőegységek által rögzített adatok időbeli összehangolása. A feladat során először beolvassuk és ábrázoljuk a szenzor, az adatgyűjtő, valamint a szinkronizációs mérések nyers adatait. A szinkronizációs pontokra illesztett lineáris időbélyeg-transzformáció segítségével az adatgyűjtő időskáláját a szenzor időskálájára alakítjuk, majd megvizsgáljuk az így kapott jelek egyezését. Az összehangolt adatokból lineáris interpolációval becsüljük a szenzorértékeket az adatgyűjtő mintavételi időpontjaiban. Az elemzést 50 Hz-es, 500 Hz-es, valamint változó amplitúdójú mérésre is elvégeztük.

1.1. feladat

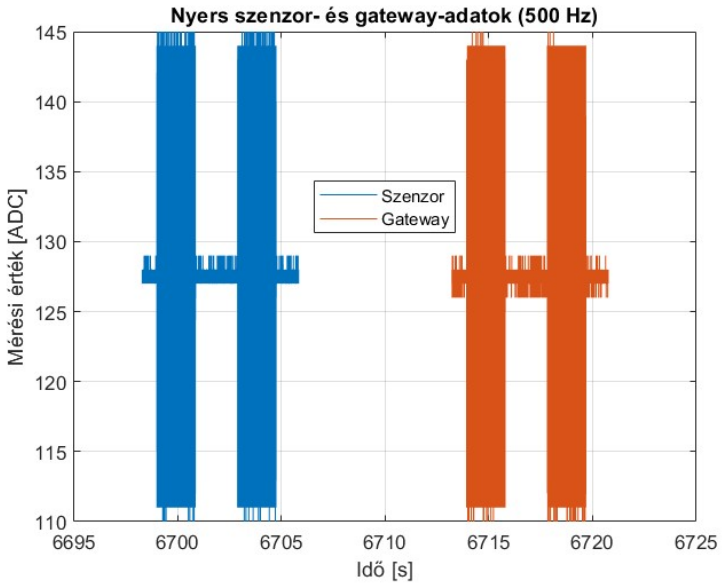
Az első feladat első pontja a mérési összeállítás elkészítése volt. Külső jelként egy függvénygenerátort használtunk, amit a Low Pass Filter bemenetére kötöttünk. Az LPS kimenetét az ellenállásokon keresztül a szenzor és a gateway bemenetére kapcsoltunk, ide csatlakoztatva egy oszcilloszkópot is, a jel valós idejű ellenőrzésének céljából. A gateway-t végül a számítógéphez csatlakozattuk, hogy a méréseket tudjuk rögzíteni és kiértékelni. A DIP kapcsolókat a mérési útmutatóban leírt módon beállítottuk, majd a jelet megvizsgáltuk az oszcilloszkópon és matlabban is. Az összeállítás az alábbi blank. ábrán látható:

1.2. feladat

A következő feladat a mérési regisztrátum készítése volt. A mérés rögzítéséhez a mérési jegyzőkönyvben leírt programot használtuk, majd a `preprocessSampleFiles.m` script futtáatása után kapott, egyszerűen feldolgozható fájlokat vizsgáltuk. A mérést elvégeztük 50 Hz-es és 500 Hz-es szinuszos jelekre is, úgy, hogy mérés közben ki és bekapcsoltuk a jelgenerátort. A jelek amplitúdóját mind a kettő esetben 2 V-ra állítottuk, ezzel kitöltve a mérési tartományt. A mérési eredmények 50 Hz-es és 500 Hz-es jelekre az alábbi blank. ábrákon láthatóak:



1. ábra. Nyers szenzoradatok (50 Hz).

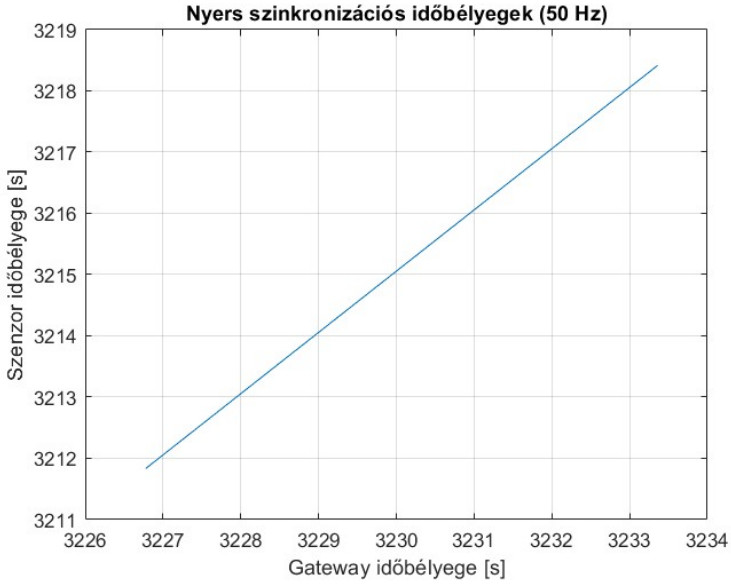


2. ábra. Nyers szenzoradatok (500 Hz).

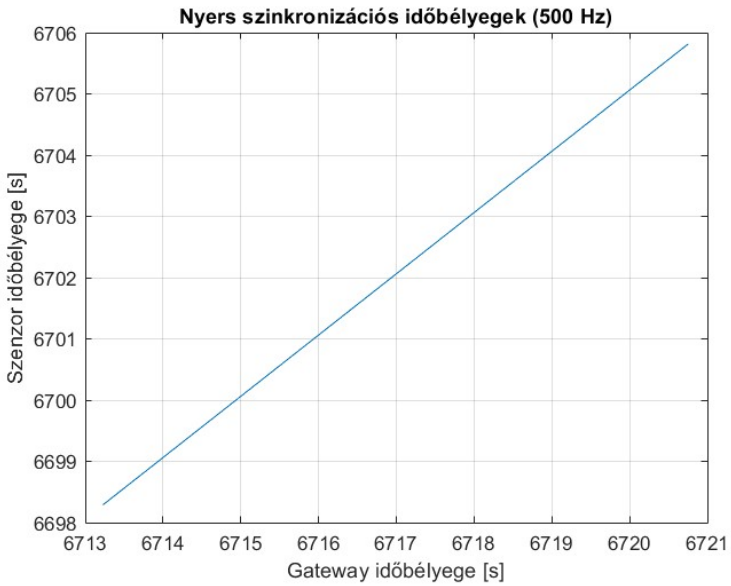
1.3. feladat

A harmadik feladatban az volt a célunk, hogy a szenzor és a gateway által rögzített jeleket időben összehangoljuk. Az időbeli eltérést az eszközöknél kezdeti időeltérés a két belső óra skáláján és a két órajel mintavételi frekvenciája közötti elrétés is okozhatta. Ennek a megoldására egy olyan lineáris transzformációt kerestünk, ami a gateway időskáláját a szenzor időskálájára alakítja.

Ehhez az `stmpSync` fájlt használtuk fel, amiben szinkronizációs időbélyeg párok szerepelek. Ez azt jelenti, hogy ezek az adatok két eszköz által rögzített, azonos eseményekhez tartozó időbélyegek. A Matlab `polyfit` függvényével ezekre az időbélyegekre lineáris függvényt illesztettünk, ami az alábbi ábrákon látható 50 Hz-es és 500 Hz-es jelekre:



3. ábra. Szinkronizációs időbélyegek (50 Hz).



4. ábra. Szinkronizációs időbélyegek (500 Hz).

A `polyfit` függvény az illesztett elsőfokú polinom együtthatóit adja vissza. A méréshez tartozó szinkronizációs pontokra az illesztés eredménye közelítőleg

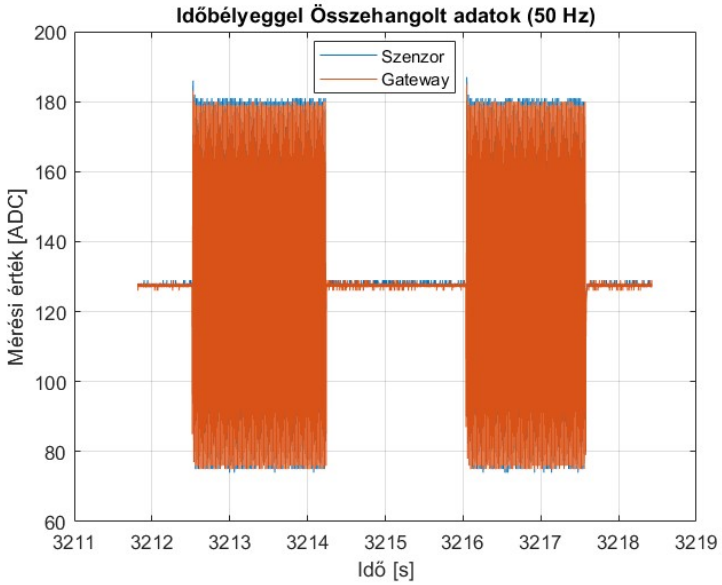
$$\texttt{idobelyegPoli} = [1, -14{,}9512] \tag{1}$$

Mivel elsőfokú polinomot illesztettünk, az első érték a meredekség, a második pedig a tengelymetszet. Ezért a gateway időbélyegét a következő összefüggéssel alakítottuk át a szenzor időskálájára:

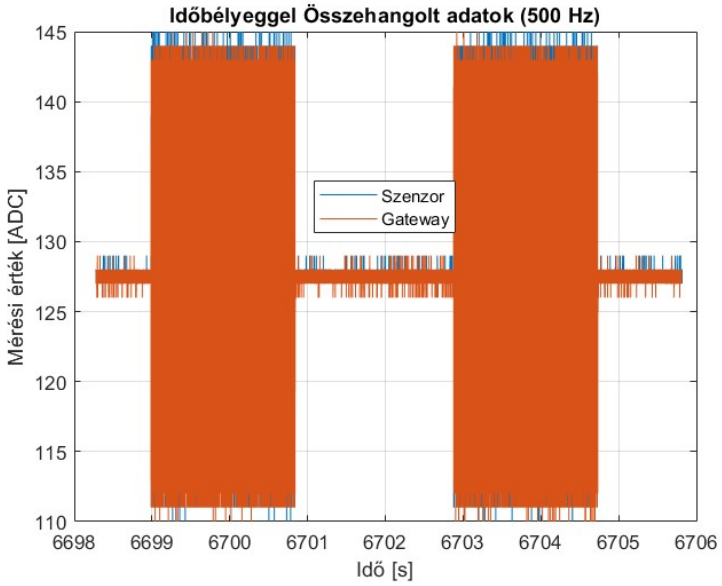
$$t_{\text{szenzor}} = 1 \cdot t_{\text{gateway}} - 14{,}9512 \tag{2}$$

A tengelymetszet pedig azt a kezdeti időeltérést írja le, amely a két eszköz időbélyegei között fennáll.

Az illesztési együtthatókat a `polyval` függvénnyel alkalmaztuk `gwySampX` minden elemére, így kiszámítva az illesztett polinom által becsült, a szenzor időskálájára átszámított időbélyegeket, amik az alábbi ábrákon láthatóak 50 Hz-es és 500 Hz-es jelek esetén:

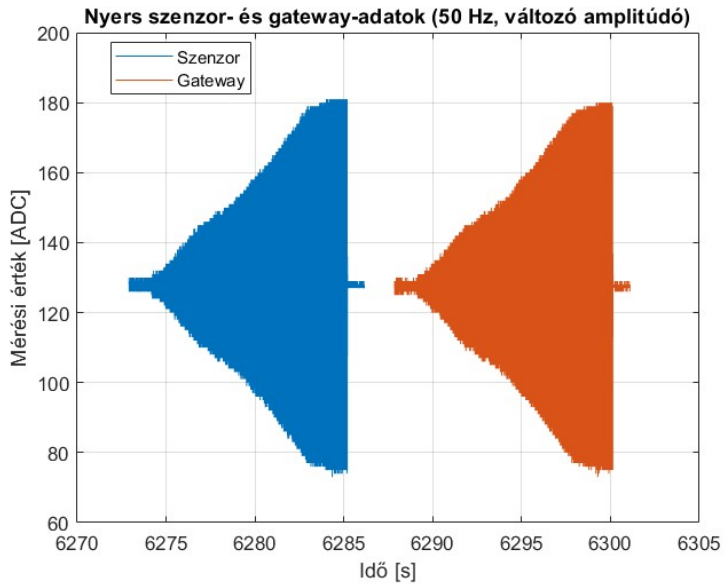


5. ábra. Időbélyeggel összehangolt jelek (50 Hz).

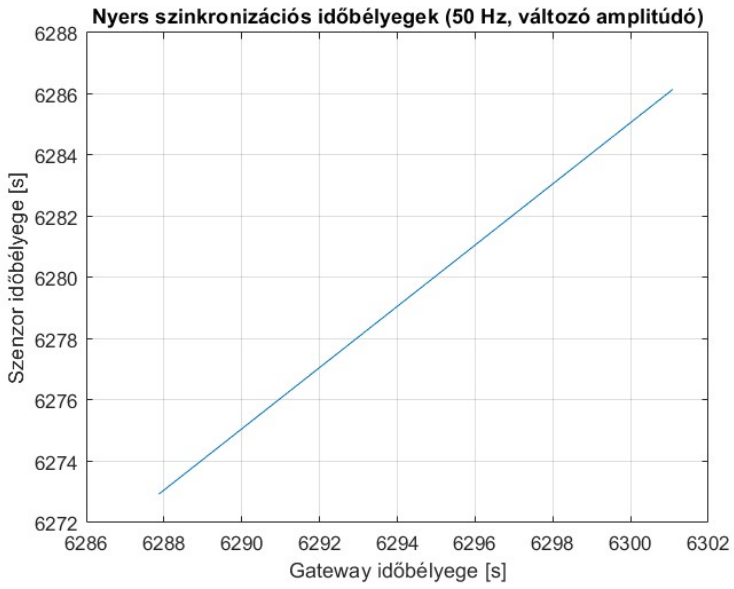


6. ábra. Időbélyeggel összehangolt jelek (500 Hz).

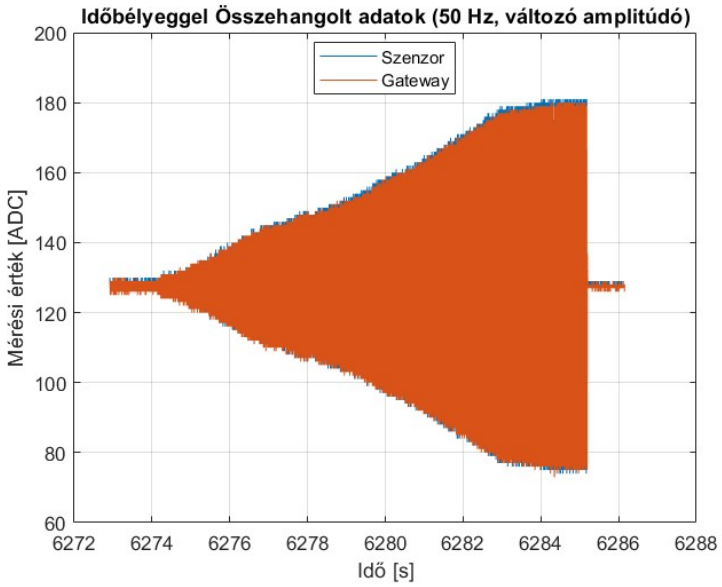
Ezt követően egy mérési regisztrátumot készítettünk, amelyben a jel amplitúdóját az indulástól kezdve folyamatosan növeltük, majd elvégeztük rá a korábbi feladatokban leírt összehangolását. Ez alatt a mérés alatt 50 Hz-es frekvencián mértünk. Az alábbi ábrákon látható a nyers mérési regisztrátum, a szinkronizációs időbélyegek, valamint az összehangolt jelek:



7. ábra. Nyers szenzor jelek változó amplitúdóval.



8. ábra. Szinkronizációs időbélyegek változó amplitúdóval.



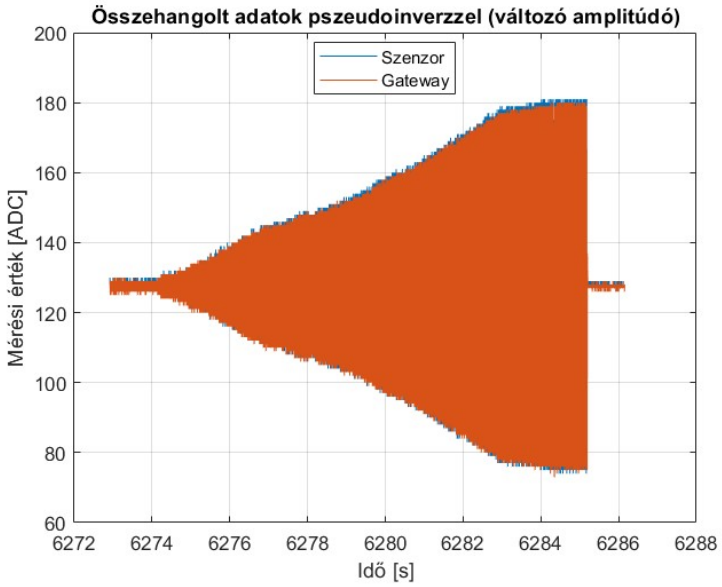
9. ábra. Időbélyeggel összehangolt jelek változó amplitúdóval.

1.4. feladat

A negyedik feladatban az előző feladatban bemutatott időbélyeg-transzformációt végeztük el újra, LS-becslés segítségével. Felépítettük az $\mathbf{X}$ mátrixot, amelynek első oszlopa a gateway szinkronizációs időbélyegeit, második oszlopa pedig egyeseket tartalmazott. Az $\mathbf{y}$ vektor a szenzor szinkronizációs időbélyegeit tartalmazta. A számításhoz a Moore–Penrose-pszeudoinverzt használtuk, amelynek LS-megoldása:

$$p = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \tag{3}$$

A MATLAB beépített függvényei között megtalálható a `pinv` függvény, amellyel ez a megoldás SVD-alapú módon számítható. A pszeudoinverzzel összehangolt jelek az alábbi ábrán láthatóak:



10. ábra. Pszeudoinverz segítségével összehangolt jelek változó amplitúdóval.

Ezt követően a gateway szinkronizációs időadatait és a gateway mérési időtengelyét 10^6 másodperccel eltoltuk, majd így ismét elvégeztük a feladatot, és vizsgáltuk a mátrix kondicionáltságát. Az első esetben a kondíciószám $1{,}0367 \cdot 10^7$ volt, az időeltolást követően pedig ez az érték $2{,}6496 \cdot 10^{11}$-re növekedett. Az ábrán ennek ellenére nem látható lényeges eltérés, mivel a lineáris transzformáció konstans tagja kompenzálja az időkoordináta közös eltolását. A nagyobb kondíciószámból azonban következik, hogy a becslés érzékenyebbé válik a mérési és kerekítési hibákra.
A probléma megoldására az időadatokat az illesztés előtt központosítottuk. Ehhez a gateway szinkronizációs időadataiból kivontuk azok átlagát, és ugyanezt az eltolást alkalmaztuk a gateway mérési időpontjaira is. Így a transzformáció eredménye nem változik, miközben a mátrix kondicionáltsága javul.

1.5. feladat

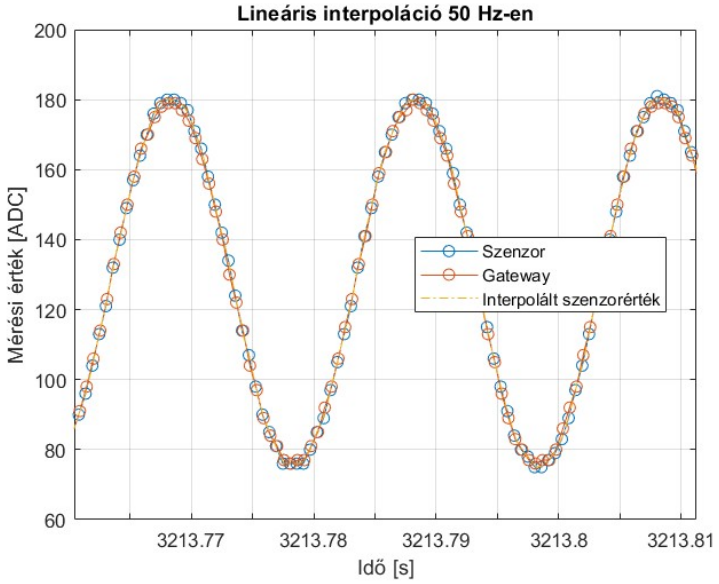
Az ötödik feladatban a szenzor mérési eredményeit a gateway mintavételi időpontjaiban kellett meghatározni lineáris interpoláció segítségével. Először a szinkronizációs pontokból lineáris időbélyegtranszformációt hajtottunk végre, a korábbi feladatokban leírt módszer segítségével. Ezután a gateway minden mintavételi időpontját átszámítottuk a szenzor időrendszerébe. Az `interp1` függvény ezekben az időpontokban a szenzor két szomszédos mintája közötti egyenest használva becsülte meg a szenzor mérési értékét, az alábbi összefüggés szerint:

$$\hat{s}(t) = s(t_i) + \frac{t - t_i}{t_{i+1} - t_i}\big(s(t_{i+1}) - s(t_i)\big), \qquad t_i \le t \le t_{i+1}. \tag{4}$$

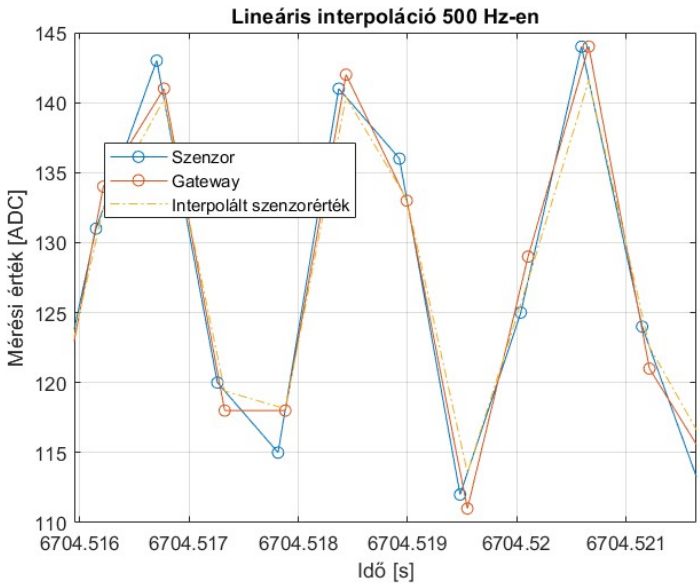
A kapott interpolált szenzorértékeket a gateway ugyanazon időpontokban mért értékeivel hasonlítottuk össze. Az összehasonlításhoz az átlagos négyzetes eltérést is kiszámítottuk:

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}\left(y_{\mathrm{gateway},i} - \hat{y}_{\mathrm{szenzor},i}\right)^2. \tag{5}$$

A programot 50 Hz-es és 500 Hz-es mérésre is lefuttattuk. Az 50 Hz-es jel esetén az MSE értéke $0{,}7255$, 500 Hz-es jel esetén pedig $1{,}9439$ volt. Az 500 Hz nagyobb változási sebessége volt a nagyobb hiba oka a második esetben. A mérési eredmények az alábbi ábrákon láthatóak, kellő nagyítással a jobb vizualizáció érdekében:



11. ábra. Lineáris interpoláció 50 Hz-en.



12. ábra. Lineáris interpoláció 500 Hz-en.

1.6. feladat

A hatodik feladatban a szenzor mérési eredményeit felmintavételezéssel sűrítettük annak érdekében, hogy a gateway mintavételi időpontjaiban pontosabban tudjuk meghatározni a szenzorjel értékét. A felmintavételezésre azért volt szükség, mert a két mérőegység nem ugyanabban az időpillanatban rögzítette a mintákat. A feldolgozást az 50 Hz-es és az 500 Hz-es mérésre is elvégeztük. A szenzorjelet tízszeresen mintavételeztük fel. Ennek során az eredeti szenzorminták közé nullákat illesztettünk, majd az így kapott jelet aluláteresztő szűrővel simítottuk.Az eredeti szenzorminták közötti átlagos időeltérésből meghatároztuk az eredeti mintavételi frekvenciát:

$$T_s = \text{mean}\left(\text{diff}(t_{\text{szenzor}})\right), \qquad f_s = \frac{1}{T_s}. \tag{6}$$

A tízszeres felmintavételezés után az új mintavételi frekvencia:

$$f_{s,\text{j}} = 10 f_s. \tag{7}$$

A nullákkal kiegészített jelet negyedrendű Butterworth-aluláteresztő szűrővel szűrtük. A szűrő levágási frekvenciáját az eredeti mintavételi frekvencia felének választottuk. A szűrést a MATLAB `filtfilt` függvényével végeztük, amely előre és visszafelé is végrehajtja a szűrést, ezért nem okoz fázistolást. A szűrés után a jelet a felmintavételezési tényezővel skáláztuk, hogy a mérési értékek amplitúdója megfelelő maradjon.

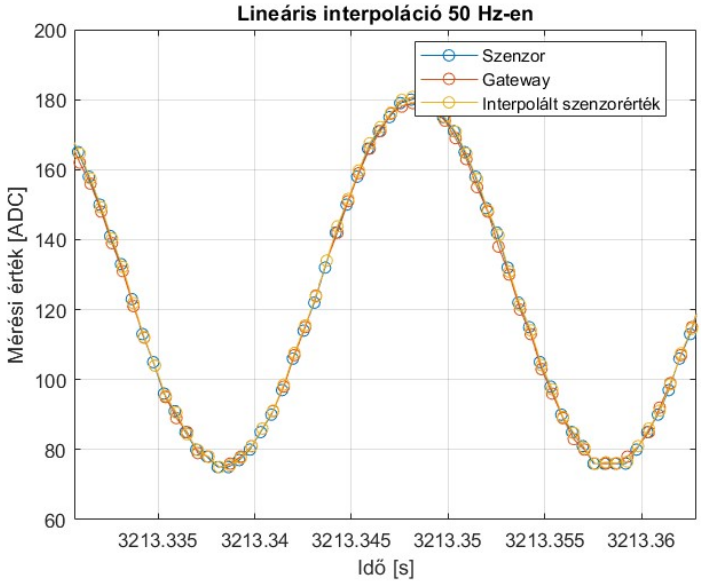
Ezt követően létrehoztuk a felmintavételezett szenzorjelhez tartozó új időtengelyt. Az időpontokat a mérés kezdő időpontjától számítottuk az új, sűrűbb mintavételi frekvenciával. Ezután a korábbi feladatokban ismertetett időbélyeg-transzformációt alkalmaztuk. A szinkronizációs időbélyegpárokra elsőfokú polinomot illesztettünk, majd a `polyval` függvénnyel a gateway minden mintavételi időpontját átszámítottuk a szenzor időskálájára.
A szenzorértékeket ezekben az időpontokban lineáris interpolációval határoztuk meg. Az interpoláció során a keresett időpontot közrefogó két szenzorminta közötti egyenest használtuk:

$$\hat{s}(t) = s(t_i) + \frac{t - t_i}{t_{i+1} - t_i}\left(s(t_{i+1}) - s(t_i)\right), \qquad t_i \le t \le t_{i+1}. \tag{8}$$

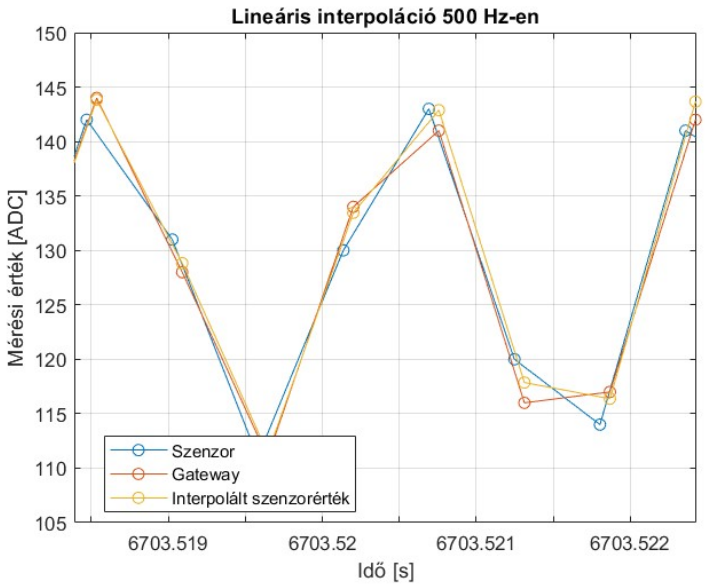
Az interpolációt a MATLAB `interp1` függvényével valósítottuk meg. Az így kapott szenzorértékeket a gateway ugyanazon időpontokban mért értékeivel hasonlítottuk össze. Az összehasonlításhoz kiszámítottuk az átlagos négyzetes eltérést:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}\left(y_{\text{gateway},i} - \hat{y}_{\text{szenzor},i}\right)^2. \tag{9}$$

Az összehasonlításból kihagytuk azokat a mintákat, amelyekhez nem tartozott érvényes interpolált szenzorérték, valamint a mérés elején és végén található 50-50 mintát. Ezzel elkerültük az időtartomány szélén fellépő interpolációs és szűrési hatások torzító hatását.
Az átlagos négyzetes eltérés értékének 50 Hz-es jel esetén $1,34037$, 500 Hz-es jel esetén pedig $1,57142$-t kaptunk. Ezek alapján megállapítható, hogy 500 Hz-es jel esetén az interpoláció eredményei jobbak, mint az 1.5. feladatban bemutatott módszer esetén. Az interpoláció eredményei az alábbi ábrákon láthatóak:



13. ábra. Lineáris interpoláció 50 Hz-en.



14. ábra. Lineáris interpoláció 500 Hz-en.

1.7. feladat

A hetedik feladatban a szenzorértékeket lokális polinomos interpolációval becsültük meg a gateway mintavételi időpontjaiban, minden gateway-mintához a hozzá legközelebb eső szenzorminta környezetében lévő 11 pontját használtuk fel. A feldolgozást az 50 Hz-es és az 500 Hz-es mérésre is elvégeztük, majd a korábbi feladatokban olvashtó időbélyeg transzformációt is elvégeztük.

Az átszámított gateway-időpontokhoz megkerestük a legközelebbi szenzormintát. Ezt a MATLAB `interp1` függvényével, a `nearest` módszer használatával valósítottuk meg. A legközelebbi szenzorminta indexe jelölte ki a lokális polinom középpontját. A lokális környezet méretét a kódban 5-nek

választottuk, így jött ki a középponttal együtt a 11 minta, amelyhez $2 \cdot 5 = 10$-ed fokú polinomot illesztettünk.

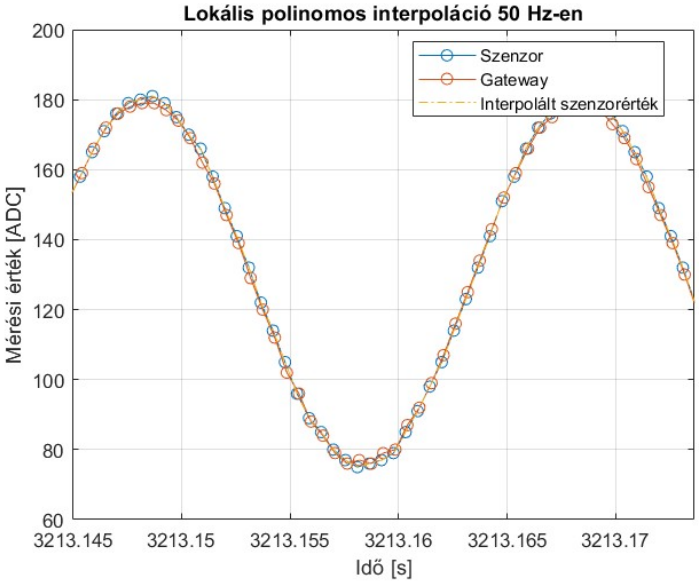
Az illesztést nem a nagy abszolút időértékekkel, hanem a gateway keresett időpontjához képesti lokális időeltérésekkel végeztük. A kiszámoltuk szenzormintákhoz tartozó időeltéréseket, majd ezeket a numerikai stabilitás javítása érdekében a legnagyobb abszolút értékkel is normáltuk. Ezután a MATLAB `polyfit` függvényével a normált lokális időkoordinátákra és a szenzorértékekre illesztettük a polinomot:

$$y_i \approx a_{10}x_i^{10} + a_9x_i^9 + \ldots + a_1x_i + a_0. \tag{10}$$

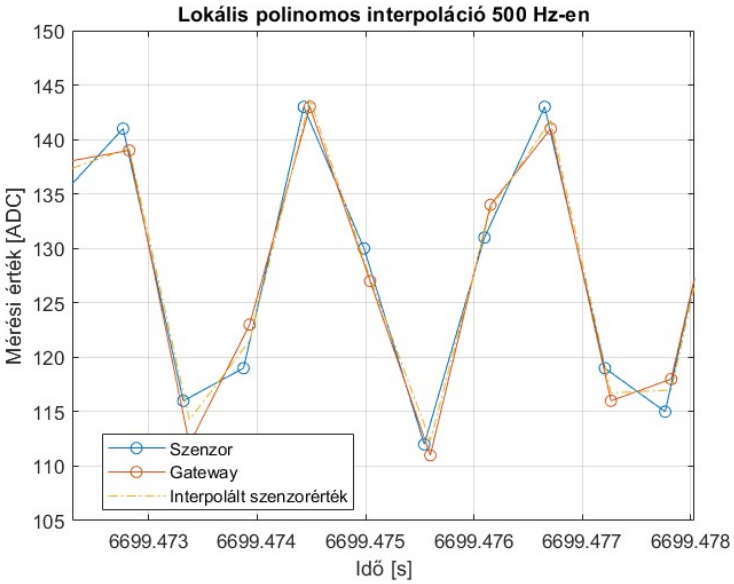
Mivel a gateway mintavételi időpontja a normált lokális időkoordinátán $x = 0$-nak felel meg, a keresett szenzorértéket az illesztett polinom $x = 0$ helyen felvett értékeként számítottuk ki. Így minden gateway mintához egy, a környező szenzorminták alakját figyelembe vevő lokális becslést kaptunk.
A mérési tartomány elején és végén nem áll rendelkezésre minden szükséges szomszédos pont, ezekhez az értékekhez a program `NaN` értéket rendelt, és az összehasonlítás során ezeket az értékeket figyelmen kívül hagytuk.
Az interpoláció minőségét a gateway mért értékei és a lokális polinommal becsült szenzorértékek közötti átlagos négyzetes eltéréssel jellemeztük:

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}\left(y_{\mathrm{gateway},i} - \hat{y}_{\mathrm{szenzor},i}\right)^2. \tag{11}$$

Az átlagos négyzetes eltérés értékének 50 Hz-es jel esetén $0,831329$-et, 500 Hz-es jel esetén pedig $0,683739$-et kaptunk. Ezek alapján megállapítható, hogy az interpoláció eredményei jobbak, mint a korábbi feladatokban bemutatott módszer esetén, mert a becslült jel lokális alakját több mérési pont alapján határozza meg,viszont a magas polinomfok és a lokális környezet nem megfelelő megválasztása oszcillációt vagy numerikai érzékenységet okozhat. Az interpoláció eredményei az alábbi ábrákon láthatóak:



15. ábra. Interpoláció 10. fokú polinommal 50 Hz-en.



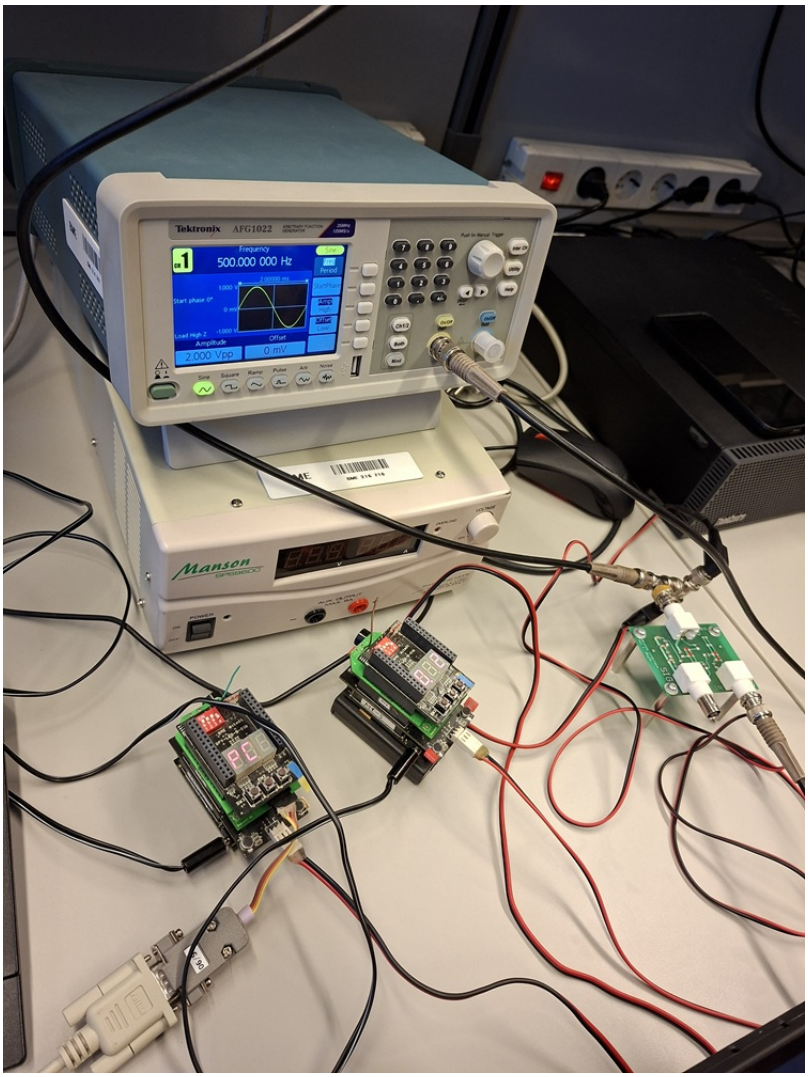
16. ábra. Interpoláció 10. fokú polinommal 500 Hz-en.

2. feladat

A második főfeladatban egy rezonátor alapú tömörítést alkalmazó mérőrendszert vizsgáltunk. A mérőrendszer elrendezése az első főfeladathoz hasonló, viszont ebben a feladatban a szenzor nem a jelet továbbítja, hanem csak annak Fourier-együtthatóit. Ezáltal csökken az átküldendő adatmennyiség, viszont a jel visszaállításához több utólagos számítás szükséges.

2.1. feladat

Az első feladatban összeállítottuk a mérési elrendezést, a szenzor nodeot a 2-es üzemmódban elindítva, hogy a Fourier-együtthatókat továbbítsa a PC felé. Ezt követően a szenzoron és a gatewayen is a megfelelő állapotba kapcsoltuk a DIP kapcsolókat. A mérési elrendezés az alábbi 17. ábra ábrán látható:

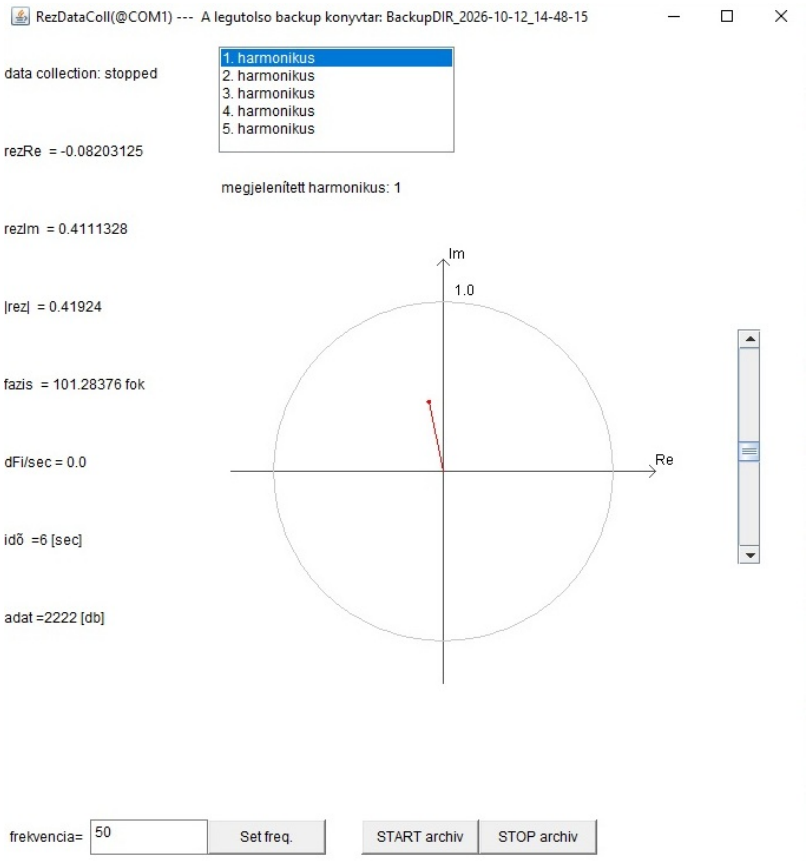


17. ábra. A mérési elrendezés.

Ezt követően a PC-n elindítottuk a mérési programot, majd a függvénygenerátoron beállítottuk a 50 Hz-es szinuszos jelet és kipróbáltuk a szoftver működését.

2.2. feladat

Az 50 Hz-es szinuszos jelet kapcsoltunk először a rendszerre. Az a) feladatban kipróbáltuk, hogy hogyan tudjuk a jel frekvenciáját meghatározni a mérési szoftver segítségével. Egy kis rezonátor frekvenciáról indulva elkezdtük növelni addig amíg az alapharmonikus fázisa konstans értéket nem mutatott. Ez 50 Hz-nél következett be amit a 18. ábra szemléltet.



18. ábra. Konstans alapharmonikus fázis.

Ezt követően a b) feladatban a MATLAB `tic` és `toc` függvényeit használva is meghatároztuk a jel frekvenciáját megközelítőleg, mivel egy ilyen szemre történő mérésnél nehéz pontosan meghatározni a jel frekvenciáját.

Végül a c) feladatban az `angle` és `unwrap` függvények segítségével meghatároztuk a jel fázisát, majd a `polyfit` függvénnyel lineáris illesztést végeztünk a fázisra. Ezt követően a meredekséget elosztva a 2π-vel megkaptuk a jel frekvenciáját, ami 50 Hz lett.

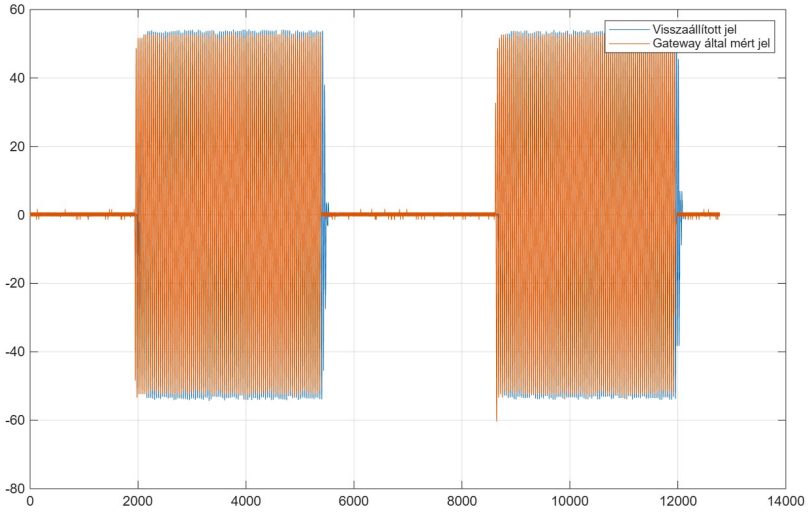
2.3. feladat

A feladat első lépéseként mérési regisztrátumot készítettünk úgy, hogy a jelgenerátort bekapcsoltuk, kikapcsoltuk, bekapcsoltuk, majd megint kikapcsoltuk. A mérés során a jelgenerátor frekvenciáját 50 Hz-en hagytuk, az amplitúdót pedig 2 V-ra állítottuk. A mérési regisztrátumot a `preprocessResonatorFiles.m`

script futtatásával dolgoztuk fel.

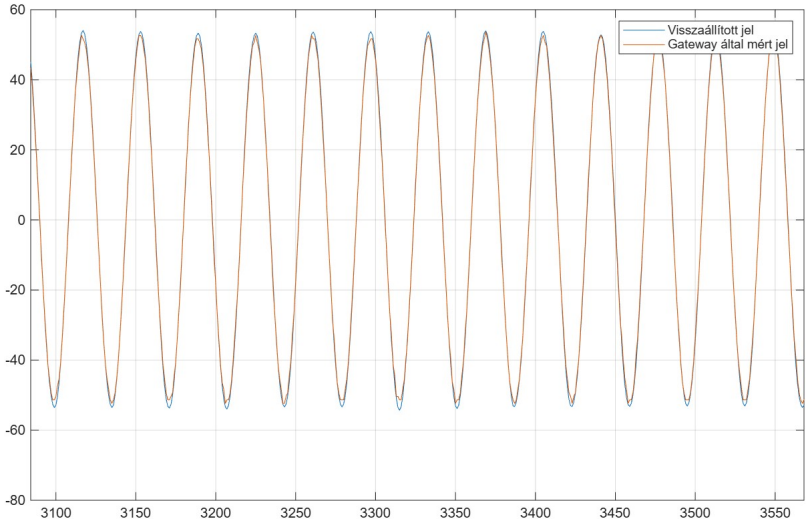
Ezt követően eltávolítottuk a gateway adatainak DC komponensét, majd a korábbi főfeladathoz hasonlóan elvégeztük az időbélyeg transzformációt, tehát átkonvertáltuk a sensor időbélyegeit a gateway időbélyegeire.

Következő feladatrész a jel visszaállítása a Fourier-együtthatókból volt. Mivel a Fourier-együtthatók sokkal ritkábban érkeznek mint a gateway mintavételezési idejei, elsőként megkellett határozni azt, hogy melyik Fourier-csomag van legközelebb az adott kiértékelési időponthoz, hogy a megfelelől Fourier-együtthatókat használhassuk a jel visszaállításához. Ezután következhetett a jelkomponensek fázisainak, majd a bázisfüggvényeknek a meghatározása, majd a jel visszaállítása. A visszaállított jelet és a gateway által mért jelet a(z) 19. ábra szemléltettük:



19. ábra. A visszaállított jel és a gateway által mért jel.

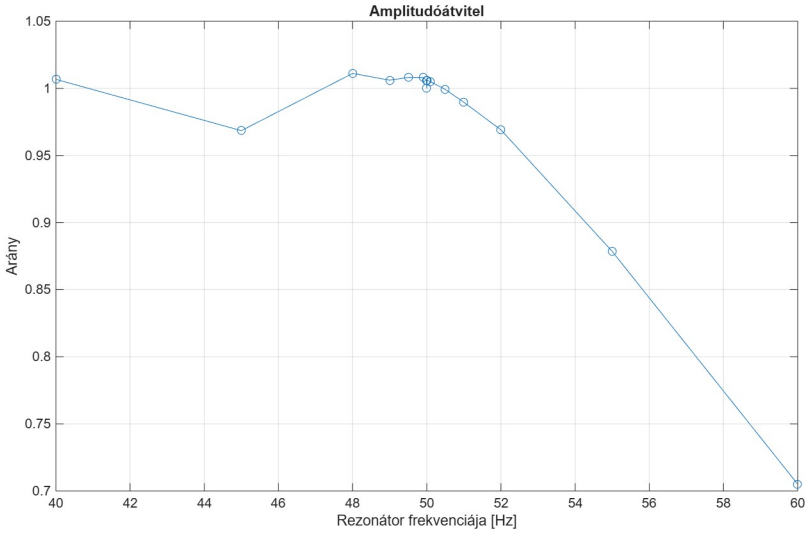
Belenagyítva a visszaállított jel és a gateway által mért jel közötti eltérést a(z) 20. ábra ábrán szemléltettük:



20. ábra. A visszaállított jel és a gateway által mért jel belenagyítva.

2.4. feladat

A feladat első lépéseként a jelgenerátor frekvenciáját 50 Hz-en hagytuk és a rezonátor frekvenciáját fokozatosan elállítottuk mindkét irányba ± 0.01 Hz, ± 0.1 Hz, ± 0.5 Hz, ± 1 Hz, ± 2 Hz, ± 5 Hz és ± 10 Hz-re. A mérés során a jelgenerátor amplitúdóját 2 V-ra állítottuk, majd a mérési regisztrátumot a `preprocessResonatorFiles.m` script futtatásával dolgoztuk fel.
Az 50 Hz-es amplitudóátvitellel normalizálva a(z) 21. ábra-n látható az amplitudóátvitel a rezonátor frekvenciájának függvényében.

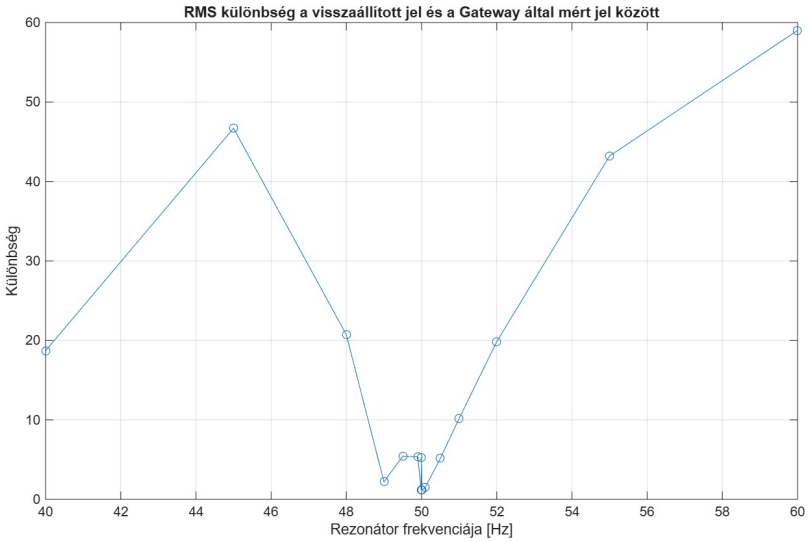


21. ábra. Az amplitudóátvitel a rezonátor frekvenciájának függvényében.

A DFT-vel összehasonlítva a módszert, az eredmény éppen ellentétes jellegű. Amíg a DFT-nél a valós frekvencián a legnagyobb az amplitudó, addig a rezonátor esetében a valós frekvencia közelében volt a legkisebb, közelében elkezdett növekedni, majd távolodva újra csökkenni. A 40 Hz-es mérést valószínleg hiba terhelheti.

2.5. feladat

Ebben a feladatban a korábbi méréseket felhasználva megvizsgáltuk az eltérést a visszaállított jel és a gateway által mért jel között. A frekvencia függvényében alakuló RMS eltéréseket a(z) 22. ábra-n szemléltettük.



22. ábra. Az eltérés a visszaállított jel és a gateway által mért jel között.

Ahogy látható, kis eltérés esetén még elfogadható hibával visszaállítható a jel.

2.6. feladat

A hatodik feladatban a gateway által sűrűn mintavételezett időtartománybeli jelből becsültük a jel Fourier-együtthatóit, majd az ezekből visszaállított jelet összehasonlítottuk a gateway által mért jellel. A feladat célja annak vizsgálata volt, hogy a rezonátoros megfigyelő képes-e a gateway-adatok alapján előállítani a szenzor Fourier együtthatóit.

A szenzor által továbbított első öt harmonikushoz tartozó Fourier-együtthatókat a a valós és képzetes részből állítottuk elő, a korábbi feladatokhoz hasonló módon.A jelek DC-komponensét a feldolgozás elején eltávolítottuk, majd időbélyeg transzformációt végeztünk, szintén a korábbi feladatok során már ismertetett módszer segítségével.

Ezután minden gateway mintához megkerestük a legközelebbi szenzor-adatcsomagot.A MATLAB `interp1` és `nearest` függvényéve. Ebből következik, hogy a szenzor által szolgáltatott együtthatók összehasonlító ábrája lépcsőzetes, mivel egy-egy szenzorcsomag több egymást követő gateway-mintához is hozzárendelésre kerül.

A Fourier-bázis fázisát a szenzor által küldött referenciafázisból számítottuk, a 2.3 feladat során már korábban felhasznált módszer segítségével, majd a meghatározott fázisokból szintén a 2.3 feladatban felhasznált módszerrel a bázisfüggvények is meghatározhatóak voltak. Ezek összefüggései az alábbi 12. és 13. egyenletekben láthatóak:

$$\varphi_1(t) = \varphi_1(t_r) + 2\pi f(t - t_r). \tag{12}$$

Az i-edik harmonikus bázisfüggvénye:

$$c_i(t) = e^{\mathrm{j}i\varphi_1(t)}, \qquad i = 1,\ldots,5. \tag{13}$$

A valós, DC-mentes jel a pozitív frekvenciájú Fourier-együtthatókból az alábbi, már korábban felhasznált összefüggés segítségével állítható elő:

$$y(t) = 2\,\mathrm{Re}\left\{\sum_{i=1}^{5} x_i(t)c_i(t)\right\}. \tag{14}$$

Az együtthatók becsléséhez a mérési útmutatóban megismert rezonátoros megfigyelőt használtuk fel. Először minden gateway-mintára létrehoztuk az aktuális komplex bázisvektort:

$$\boldsymbol{c}_n = \left[e^{\mathrm{j}\varphi_n}, e^{\mathrm{j}2\varphi_n}, \ldots, e^{\mathrm{j}5\varphi_n}\right]^T. \tag{15}$$

A jegyzőkönyv (8)–(10) egyenletei alapján először kiszámítottuk a Fourier-együtthatók által becsült jelet:

$$y'_n = 2\,\mathrm{Re}\left\{\boldsymbol{c}_n^T \hat{\boldsymbol{x}}_n\right\}. \tag{16}$$

Ezután meghatároztuk a gateway-mérés és a becslés különbségét:

$$e_n = y_n - y'_n. \tag{17}$$

A becslési hiba alapján frissítettük a komplex Fourier-együtthatók vektorát:

$$\hat{\boldsymbol{x}}_{n+1} = \hat{\boldsymbol{x}}_n + \alpha e_n \boldsymbol{c}_n^*. \tag{18}$$

A számítások során a bátorsági tényezőt

$$\alpha = 0{,}01 \tag{19}$$

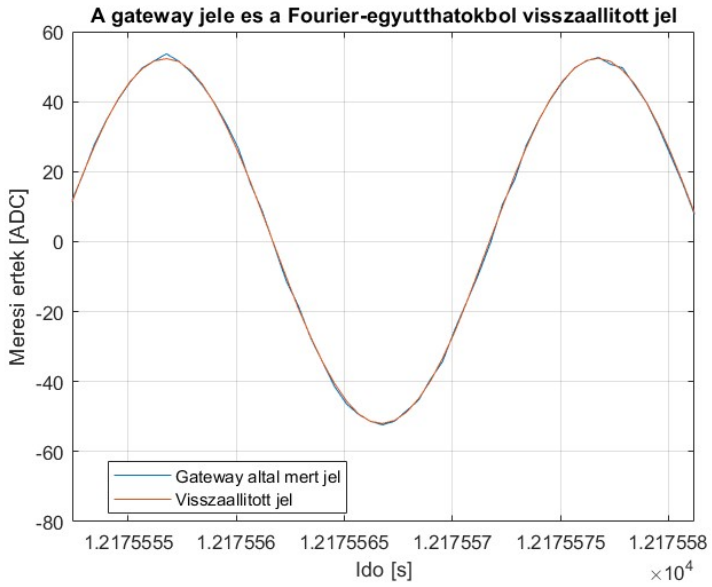
értékre állítottuk. Erre a döntésre kísérleti úton jutottunk, ugyanis ezzel az értékkel kaptuk a legkisebb RMS eltérést. A bátorsági tényező növelése esetén a megfigyelő gyorsabban reagált a jel amplitúdójának változására, viszont a felfutó és lecsengő tranziens szakaszokban túllövés jelent meg.

A Fourier-együtthatók kezdeti becslését nulláról indítottuk, emiatt a jel be- és kikapcsolásakor a rekonstruált jel nem azonnal éri el a gateway-jel amplitúdóját, hanem fokozatosan áll be. Ez a rezonátoros megfigyelő beállási tranziensből származó eltérés.

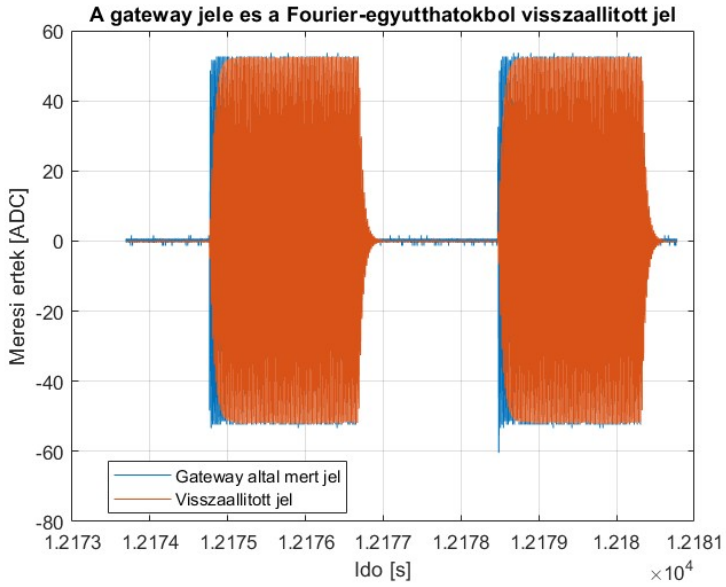
Az algoritmus eredményeit vizuális kettő ábrán vizsgáltuk, a 23. és 24. ábrában a gateway által mért időtartománybeli jelet és az öt Fourier-együtthatóból visszaállított jelet vetettük össze, a 25. ábránpedig a gateway-adatokból becsült, valamint a szenzor által szolgáltatott Fourier-együtthatók

amplitúdóját hasonlítottuk össze.

Az alábbi 24. ábrán látható a fentebb említett eltérés a felfutási és lecsengési tranziens esetén, viszont a 23. ábrán nagyítás után az állandósult szakaszon csak minimális eltérés látható a két jel között.

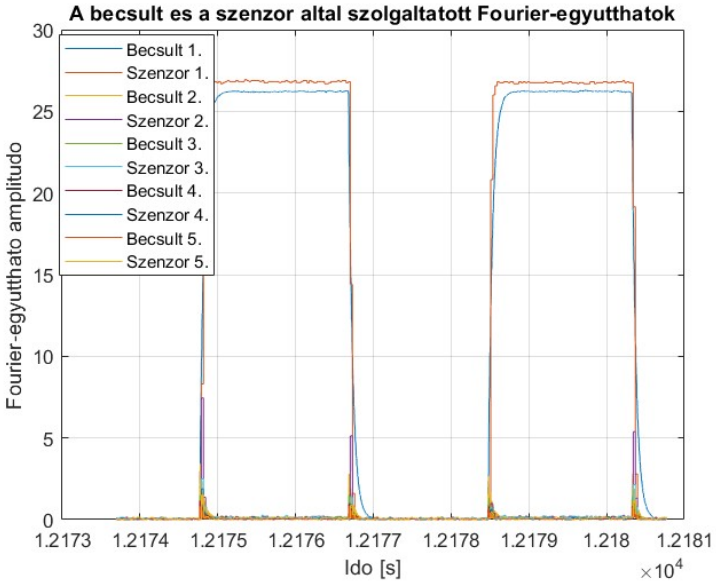


23. ábra. Gateway jel és visszaállított jel belenagyítva.



24. ábra. Gateway jel és visszaállított jel.

A 25. ábrán a gateway-adatokból becsült, valamint a szenzor által szolgáltatott Fourier-együtthatók amplitúdójának összehasonlítása látható. Az első harmonikus amplitúdója volt a legnagyobb, ezért annak abszolút hibája is nagyobb lett, illetve szintén a fentebb leírt tranzienseknél jelentkező különbség is látható.



25. ábra. Becsült és szenzor által szolgáltatott Fourier-együtthatók amplitúdója.

Az összehasonlítást a komplex együtthatók és a gateway-jel és a visszaállított jel közötti eltérést RMS értékével számítottuk.

A mérés során a gateway-jel és a visszaállított jel RMS eltérése 4,74589 volt. Az egyes harmonikusokra kapott RMS eltérések:

Harmonikus	RMS eltérés
1.	1,51301
2.	0,631144
3.	0,255675
4.	0,1732
5.	0,160554

(20)

Az eredmények alapján megállapítható, hogy a rezonátoros megfigyelő az állandósult szakaszban megfelelően közelíti a gateway által mért jelet. Az eltérés döntő része a jel ki- és bekapcsolásakor jelentkező tranziens szakaszokból, a becslő beállási idejéből, a mérési zajból, valamint a szenzor és a gateway eltérő mintavételi időpontjaiból származhat.

Függelék

MATLAB kódok

Az 1.1-4 mérés MATLAB-kódja

```
%%TODO
% Egészben javítani a meredekséget

%% Clearing wokspace
clear;
clf;
clc;
close all;

%% Loading measurement files and parameters for 50 [Hz] measurement
gwySampFull = load("Measurements/ElsoFeladatteszt/pre_gwySampFull.dat");
sensorDataFull = load("Measurements/ElsoFeladatteszt/pre_sensorDataFull.dat");
stmpSync = load("Measurements/ElsoFeladatteszt/pre_stmpSync.dat");

sensorDataX = sensorDataFull(:, 1);
sensorDataY = sensorDataFull(:, 2);
clearvars sensorDataFull;

gwySampX = gwySampFull(:, 1);
gwySampY = gwySampFull(:, 2);
clearvars gwySampFull;

stmpSyncGwy = stmpSync(:, 1);
stmpSyncSens = stmpSync(:, 2);
clearvars stmpSync;

%% Plotting the raw data
figure;
plot(sensorDataX, sensorDataY);
hold on;
plot(gwySampX, gwySampY);
xlabel("Idő [s]");
ylabel("Mérési érték [ADC]");
title("Nyers szenzor- és gateway-adatok (50 Hz)");
legend("Szenzor", "Gateway", "Location", "best");
grid on;
```

```
37
38 figure;
39 plot(stmpSyncGwy, stmpSyncSens);
40 xlabel("Gateway időbélyege [s]");
41 ylabel("Szenzor időbélyege [s]");
42 title("Nyers szinkronizációs időbélyegek (50 Hz)");
43 grid on;
44
45 %% Időbélyeg-transzformáció
46 idobelyegPoli = polyfit(stmpSyncGwy, stmpSyncSens, 1);
47 GwyToSensorTime  = polyval(idobelyegPoli, gwySampX);
48
49 %% Plot offset data
50 figure;
51 plot(sensorDataX, sensorDataY);
52 hold on;
53 plot(GwyToSensorTime , gwySampY);
54 xlabel("Idő [s]");
55 ylabel("Mérési érték [ADC]");
56 title("Időbélyeggel összehangolt adatok (50 Hz)");
57 legend("Szenzor", "Gateway", "Location", "best");
58 grid on;
59
60 %% Clearing workspace
61 clearvars;
62
63 %% Loading measurement files and parameters for 500 [Hz] measurement
64 gwySampFull = load("Measurements/ElsoFeladat500Hz/pre_gwySampFull.dat")
      ;
65 sensorDataFull = load("Measurements/ElsoFeladat500Hz/pre_sensorDataFull
      .dat");
66 stmpSync = load("Measurements/ElsoFeladat500Hz/pre_stmpSync.dat");
67
68 sensorDataX = sensorDataFull(:, 1);
69 sensorDataY = sensorDataFull(:, 2);
70 clearvars sensorDataFull;
71
72 gwySampX = gwySampFull(:, 1);
73 gwySampY = gwySampFull(:, 2);
74 clearvars gwySampFull;
75
76 stmpSyncGwy = stmpSync(:, 1);
77 stmpSyncSens = stmpSync(:, 2);
```

```
78 clearvars stmpSync;
79
80 %% Plotting the raw data
81 figure;
82 plot(sensorDataX, sensorDataY);
83 hold on;
84 plot(gwySampX, gwySampY);
85 xlabel("Idő [s]");
86 ylabel("Mérési érték [ADC]");
87 title("Nyers szenzor- és gateway-adatok (500 Hz)");
88 legend("Szenzor", "Gateway", "Location", "best");
89 grid on;
90
91 figure;
92 plot(stmpSyncGwy, stmpSyncSens);
93 xlabel("Gateway időbélyege [s]");
94 ylabel("Szenzor időbélyege [s]");
95 title("Nyers szinkronizációs időbélyegek (500 Hz)");
96 grid on;
97
98 %% Időbélyeg-transzformáció
99 idobelyegPoli = polyfit(stmpSyncGwy, stmpSyncSens, 1);
100 GwyToSensorTime  = polyval(idobelyegPoli, gwySampX);
101
102 %% Plot offset data
103 figure;
104 plot(sensorDataX, sensorDataY);
105 hold on;
106 plot(GwyToSensorTime , gwySampY);
107 xlabel("Idő [s]");
108 ylabel("Mérési érték [ADC]");
109 title("Időbélyeggel összehangolt adatok (500 Hz)");
110 legend("Szenzor", "Gateway", "Location", "best");
111 grid on;
112
113 %% Clearing workspace
114 clearvars;
115
116 %% Loading measurement files and parameters for 50 [Hz] measurement
       with changing amplitudes
117 gwySampFull = load("Measurements/ElsoFeladat50HzChangingAmplitudes/
       pre_gwySampFull.dat");
```

```
118 sensorDataFull = load("Measurements/ElsoFeladat50HzChangingAmplitudes/
       pre_sensorDataFull.dat");
119 stmpSync = load("Measurements/ElsoFeladat50HzChangingAmplitudes/
       pre_stmpSync.dat");
120
121 sensorDataX = sensorDataFull(:, 1);
122 sensorDataY = sensorDataFull(:, 2);
123 clearvars sensorDataFull;
124
125 gwySampX = gwySampFull(:, 1);
126 gwySampY = gwySampFull(:, 2);
127 clearvars gwySampFull;
128
129 stmpSyncGwy = stmpSync(:, 1);
130 stmpSyncSens = stmpSync(:, 2);
131 clearvars stmpSync;
132
133 %% Plotting the raw data
134 figure;
135 plot(sensorDataX, sensorDataY);
136 hold on;
137 plot(gwySampX, gwySampY);
138 xlabel("Idő [s]");
139 ylabel("Mérési érték [ADC]");
140 title("Nyers szenzor- és gateway-adatok (50 Hz, változó amplitúdó)");
141 legend("Szenzor", "Gateway", "Location", "best");
142 grid on;
143
144 figure;
145 plot(stmpSyncGwy, stmpSyncSens);
146 xlabel("Gateway időbélyege [s]");
147 ylabel("Szenzor időbélyege [s]");
148 title("Nyers szinkronizációs időbélyegek (50 Hz, változó amplitúdó)");
149 grid on;
150
151 %% Időbélyeg-transzformáció
152 idobelyegPoli = polyfit(stmpSyncGwy, stmpSyncSens, 1);
153 GwyToSensorTime = polyval(idobelyegPoli, gwySampX);
154
155 %% Plot offset data
156 figure;
157 plot(sensorDataX, sensorDataY);
158 hold on;
```

```
159 plot(GwyToSensorTime , gwySampY);
160 xlabel("Idő [s]");
161 ylabel("Mérési érték [ADC]");
162 title("Időbélyeggel összehangolt adatok (50 Hz, változó amplitúdó)");
163 legend("Szenzor", "Gateway", "Location", "best");
164 grid on;
165
166 %% Pseudoinvers calc
167
168 X = [stmpSyncGwy, ones(size(stmpSyncGwy))];
169 Y = stmpSyncSens;
170 cond(X)
171 idobelyegPoliPseudo = pinv(X) * Y;
172 GwyToSensorTime_pseudo = polyval(idobelyegPoliPseudo, gwySampX);
173
174 %% Plot offset data
175 figure;
176 plot(sensorDataX, sensorDataY);
177 hold on;
178 plot(GwyToSensorTime_pseudo, gwySampY);
179 xlabel("Idő [s]");
180 ylabel("Mérési érték [ADC]");
181 title("Összehangolt adatok pszeudoinverzzel (50 Hz, változó amplitúdó)
      ");
182 legend("Szenzor", "Gateway", "Location", "best");
183 grid on;
184
185 %% Szinkronizációs pont eltolás
186 stmpSyncGwy = stmpSyncGwy + 10^6;
187 gwySampX_shifted = gwySampX + 10^6;
188
189 %% Pseudoinvers calc
190
191 Y = stmpSyncSens;
192 X_shifted = [stmpSyncGwy, ones(size(stmpSyncGwy))];
193 cond(X_shifted)
194
195 % Az adatok központosítása javítja a mátrix kondicionáltságát.
196 timeMean = mean(stmpSyncGwy);
197 X_centered = [stmpSyncGwy - timeMean, ones(size(stmpSyncGwy))];
198 cond(X_centered)
199 idobelyegPoliPseudo = pinv(X_centered) * Y;
```

```
GwyToSensorTime_pseudo = polyval(idobelyegPoliPseudo, gwySampX_shifted
    - timeMean);

%% Plot offset data
figure;
plot(sensorDataX, sensorDataY);
hold on;
plot(GwyToSensorTime_pseudo, gwySampY);
xlabel("Idő [s]");
ylabel("Mérési érték [ADC]");
title("Pszeudoinverzes összehangolás 10^6 s időeltolással");
legend("Szenzor", "Gateway", "Location", "best");
grid on;
```

Az 1.5 mérés MATLAB-kódja

```
%% Az 1.5 feladat: lineáris interpoláció
clear;
clf;
clc;
close all;

frequencies = [50, 500];
measurementFolders = ["ElsoFeladat50Hz", "ElsoFeladat500Hz"];

for k = 1:numel(frequencies)
    frequency = frequencies(k);
    measurementFolder = measurementFolders(k);

    % A mérési adatok betöltése.
    gwySampFull = load(fullfile("Measurements", measurementFolder, ...
        "pre_gwySampFull.dat"));
    sensorDataFull = load(fullfile("Measurements", measurementFolder,
       ...
        "pre_sensorDataFull.dat"));
    stmpSync = load(fullfile("Measurements", measurementFolder, ...
        "pre_stmpSync.dat"));

    sensorDataX = sensorDataFull(:, 1);
    sensorDataY = sensorDataFull(:, 2);
    gwySampX = gwySampFull(:, 1);
    gwySampY = gwySampFull(:, 2);

    % A gateway idejét a szenzor órájára transzformáljuk.
```

```
    idobelyegPoli = polyfit(stmpSync(:, 1), stmpSync(:, 2), 1);
    gatewayTimeInSensorClock = polyval(idobelyegPoli, gwySampX);

    % A szenzor jelét a gateway mintavételi időpontjaiban becsüljük.
    sensorDataYInterpolated = interp1( ...
        sensorDataX, sensorDataY, gatewayTimeInSensorClock, "linear");

    validSamples = ~isnan(sensorDataYInterpolated);
    squaredError = mean((gwySampY(validSamples) ...
        - sensorDataYInterpolated(validSamples)).^2);
    fprintf("%d Hz: átlagos négyzetes eltérés = %.6g\n", ...
        frequency, squaredError);

    figure;
    plot(sensorDataX, sensorDataY, "-o");
    hold on;
    plot(gatewayTimeInSensorClock, gwySampY, "-o");
    plot(gatewayTimeInSensorClock, sensorDataYInterpolated, "-.");
    xlabel("Idő [s]");
    ylabel("Mérési érték [ADC]");
    title(sprintf("Lineáris interpoláció %d Hz-en", frequency));
    legend("Szenzor", "Gateway", ...
        "Interpolált szenzorérték", "Location", "best");
    grid on;
end
```

Az 1.6 mérés MATLAB-kódja

```
%% Az 1.6 feladat: felmintavételezés
%% Clearing wokspace
clear;
clf;
clc;
close all;


frequencies = [50, 500];
measurementFolders = ["ElsoFeladat50Hz", "ElsoFeladat500Hz"];

for k = 1:numel(frequencies)
    frequency = frequencies(k);
    measurementFolder = measurementFolders(k);

    %% Loading measurement files and parameters
```

```
17      gwySampFull = load(fullfile("Measurements", measurementFolder, "
            pre_gwySampFull.dat"));
18      sensorDataFull = load(fullfile("Measurements", measurementFolder, "
            pre_sensorDataFull.dat"));
19      stmpSync = load(fullfile("Measurements", measurementFolder, "
            pre_stmpSync.dat"));
20
21      sensorDataX = sensorDataFull(:, 1);
22      sensorDataY = sensorDataFull(:, 2);
23      gwySampX = gwySampFull(:, 1);
24      gwySampY = gwySampFull(:, 2);
25
26      stmpSyncGwy = stmpSync(:, 1);
27      stmpSyncSens = stmpSync(:, 2);
28
29      %% Upsampling the sensor data
30
31      upSampNum = 10;
32      Ts = mean(diff(sensorDataX));
33      fs = 1 / Ts;
34      fsNew = fs * upSampNum;
35
36      yUp = zeros(upSampNum * (numel(sensorDataY) - 1) + 1, 1);
37      yUp(1:upSampNum:end) = sensorDataY;
38
39      cutoffFrequency = fs / 2;
40      [b, a] = butter(4, cutoffFrequency / (fsNew / 2), "low");
41
42      upSampledSensorDataY = filtfilt(b, a, yUp);
43
44      upSampledSensorDataY = upSampNum * upSampledSensorDataY;
45
46      upSampledSensorDataX = sensorDataX(1) + (0:numel(yUp) - 1)' / fsNew
            ;
47
48      %% Interpolation
49      idobelyegPoli = polyfit(stmpSyncGwy, stmpSyncSens, 1);
50      gatewayTimeInSensorClock = polyval(idobelyegPoli, gwySampX);
51      sensorDataYInterpolated = interp1(upSampledSensorDataX,
            upSampledSensorDataY, gatewayTimeInSensorClock, 'linear');
52
53      validSamples = ~isnan(sensorDataYInterpolated);
54
```

```
    edgeSamples = 50;
    comparisonSamples = validSamples;
    comparisonSamples(1:edgeSamples) = false;
    comparisonSamples(end-edgeSamples + 1:end) = false;

    squaredError = mean((gwySampY(comparisonSamples) - ...
        sensorDataYInterpolated(comparisonSamples)).^2);
    fprintf("%d Hz: átlagos négyzetes eltérés = %.6g\n", frequency,
       squaredError);

    %% Plotting
    figure;
    plot(sensorDataX, sensorDataY, "-o");
    hold on;
    plot(gatewayTimeInSensorClock(comparisonSamples), ...
        gwySampY(comparisonSamples), "-o");
    plot(gatewayTimeInSensorClock(comparisonSamples), ...
        sensorDataYInterpolated(comparisonSamples), "-o");
    xlabel("Idő [s]");
    ylabel("Mérési érték [ADC]");
    title(sprintf("Lineáris interpoláció %d Hz-en", frequency));
    legend("Szenzor", "Gateway", ...
        "Interpolált szenzorérték", "Location", "best");
    grid on;
end
```

Az 1.7 mérés MATLAB-kódja

```
%% Az 1.7 feladat: lokális polinomos interpoláció
clear;
clf;
clc;
close all;

frequencies = [50, 500];
measurementFolders = ["ElsoFeladat50Hz", "ElsoFeladat500Hz"];
L = 5;
polynomialDegree = 2 * L;

for k = 1:numel(frequencies)
    frequency = frequencies(k);
    measurementFolder = measurementFolders(k);

    % A mérési adatok betöltése.
```

```
    gwySampFull = load(fullfile("Measurements", measurementFolder, ...
        "pre_gwySampFull.dat"));
    sensorDataFull = load(fullfile("Measurements", measurementFolder,
       ...
        "pre_sensorDataFull.dat"));
    stmpSync = load(fullfile("Measurements", measurementFolder, ...
        "pre_stmpSync.dat"));

    sensorDataX = sensorDataFull(:, 1);
    sensorDataY = sensorDataFull(:, 2);
    gwySampX = gwySampFull(:, 1);
    gwySampY = gwySampFull(:, 2);

    % A gateway idejét a szenzor órájára transzformáljuk.
    idobelyegPoli = polyfit(stmpSync(:, 1), stmpSync(:, 2), 1);
    gatewayTimeInSensorClock = polyval(idobelyegPoli, gwySampX);

    % Minden gateway mintához megkeressük a legközelebbi szenzormintát.
    nearestSensorIndex = nan(size(gatewayTimeInSensorClock));
    inSensorRange = gatewayTimeInSensorClock >= sensorDataX(1) ...
        & gatewayTimeInSensorClock <= sensorDataX(end);
    nearestSensorIndex(inSensorRange) = interp1( ...
        sensorDataX, (1:numel(sensorDataX))', ...
        gatewayTimeInSensorClock(inSensorRange), "nearest");

    % Lokális, 2L+1 pontból álló polinommal becsüljük az értéket.
    sensorDataYInterpolated = nan(size(gatewayTimeInSensorClock));
    validSamples = false(size(gatewayTimeInSensorClock));
    for sampleIndex = find(inSensorRange)'
        centerIndex = nearestSensorIndex(sampleIndex);
        if centerIndex - L < 1 || centerIndex + L > numel(sensorDataX)
            continue;
        end

        localIndices = (centerIndex - L):(centerIndex + L);
        localTimeOffsets = sensorDataX(localIndices) ...
            - gatewayTimeInSensorClock(sampleIndex);
        localTimeScale = max(abs(localTimeOffsets));
        localPolynomial = polyfit( ...
            localTimeOffsets / localTimeScale, ...
            sensorDataY(localIndices), polynomialDegree);
        sensorDataYInterpolated(sampleIndex) = polyval(localPolynomial,
            0);
```

```
58          validSamples(sampleIndex) = true;
59      end
60
61      squaredError = mean((gwySampY(validSamples) ...
62          - sensorDataYInterpolated(validSamples)).^2);
63      fprintf("%d Hz (L = %d, %d. fokú polinom): átlagos négyzetes eltéré
            s = %.6g\n", ...
64          frequency, L, polynomialDegree, squaredError);
65
66      figure;
67      plot(sensorDataX, sensorDataY, "-o");
68      hold on;
69      plot(gatewayTimeInSensorClock(validSamples), gwySampY(validSamples)
            , "-o");
70      plot(gatewayTimeInSensorClock(validSamples), ...
71          sensorDataYInterpolated(validSamples), "-.");
72      xlabel("Idő [s]");
73      ylabel("Mérési érték [ADC]");
74      title(sprintf("Lokális polinomos interpoláció %d Hz-en", frequency)
            );
75      legend("Szenzor", "Gateway", ...
76          "Interpolált szenzorérték", "Location", "best");
77      grid on;
78  end
```

Az 2.2 mérés MATLAB-kódja

```
1   %% Clearing wokspace
2   clear;
3   clf;
4   clc;
5   close all;
6
7   %% Loading measurement files and parameters for 50 [Hz] measurement
8   gwySampFull = load("Measurements/MasodikFeladatJel50HzProgi50Hz/
        pre_gwySampFull.dat");
9   sensorDataFull = load("Measurements/MasodikFeladatJel50HzProgi50Hz/
        pre_sensorResonator.dat");
10  stmpSync = load("Measurements/MasodikFeladatJel50HzProgi50Hz/
        pre_stmpSync.dat");
11  signalFreq = load("Measurements/MasodikFeladatJel50HzProgi50Hz/
        signalFreq.dat");
12
13  sensorDataID = sensorDataFull(:, 1);
```

```
sensorDataTime = sensorDataFull(:, 2);
sensorDataPhi = sensorDataFull(:, 3);
sensorDataFirstHarmonic = sensorDataFull(:, 4) + 1i * sensorDataFull(:, 9);
sensorDataSecondHarmonic = sensorDataFull(:, 5) + 1i * sensorDataFull(:, 10);
sensorDataThirdHarmonic = sensorDataFull(:, 6) + 1i * sensorDataFull(:, 11);
sensorDataFourthHarmonic = sensorDataFull(:, 7) + 1i * sensorDataFull(:, 12);
sensorDataFifthHarmonic = sensorDataFull(:, 8) + 1i * sensorDataFull(:, 13);
clearvars sensorDataFull;

gwySampX = gwySampFull(:, 1);
gwySampY = gwySampFull(:, 2);
clearvars gwySampFull;

stmpSyncGwy = stmpSync(:, 1);
stmpSyncSens = stmpSync(:, 2);
clearvars stmpSync;

%% c, feladat
Phase = angle(sensorDataFirstHarmonic);
Phase = unwrap(Phase);
phasePoly = polyfit(sensorDataTime, Phase, 1);
gradient = phasePoly(1) / (2* pi);
```

Az 2.3 mérés MATLAB-kódja

```
%% Clearing wokspace
clear;
clf;
clc;
close all;

%% Loading measurement files and parameters for 50 [Hz] measurement
gwySampFull = load("Measurements/
   MasodikFeladatJel50HzKiBeKapcsoltProgi50Hz/pre_gwySampFull.dat");
sensorDataFull = load("Measurements/
   MasodikFeladatJel50HzKiBeKapcsoltProgi50Hz/pre_sensorResonator.dat")
   ;
stmpSync = load("Measurements/
   MasodikFeladatJel50HzKiBeKapcsoltProgi50Hz/pre_stmpSync.dat");
```

```
signalFreq = load("Measurements/
   MasodikFeladatJel50HzKiBeKapcsoltProgi50Hz/signalFreq.dat");

sensorDataID = sensorDataFull(:, 1);
sensorDataTime = sensorDataFull(:, 2);
sensorDataPhi = sensorDataFull(:, 3);
sensorDataFirstHarmonic = sensorDataFull(:, 4) + 1i * sensorDataFull(:,
    9);
sensorDataSecondHarmonic = sensorDataFull(:, 5) + 1i * sensorDataFull
   (:, 10);
sensorDataThirdHarmonic = sensorDataFull(:, 6) + 1i * sensorDataFull(:,
    11);
sensorDataFourthHarmonic = sensorDataFull(:, 7) + 1i * sensorDataFull
   (:, 12);
sensorDataFifthHarmonic = sensorDataFull(:, 8) + 1i * sensorDataFull(:,
    13);
clearvars sensorDataFull;

gwySampX = gwySampFull(:, 1);
gwySampY = gwySampFull(:, 2);
clearvars gwySampFull;

stmpSyncGwy = stmpSync(:, 1);
stmpSyncSens = stmpSync(:, 2);
clearvars stmpSync;

%% Gateway átlag kivonás
gwySampY = gwySampY - mean(gwySampY);

%% Időbélyeg transzformáció
idobelyegPoli = polyfit(stmpSyncSens, stmpSyncGwy, 1);
sensorToGwyTime  = polyval(idobelyegPoli, sensorDataTime);

evalTimes = zeros(length(gwySampX), 1);
evalValues = zeros(length(gwySampX), 1);
evalDeltaT = zeros(length(gwySampX), 1);

for i = 1:length(gwySampX)
    minVal = 100000;
    minDeltaT = 0;
    minID = 0;
    for j = 1:length(sensorToGwyTime)
        if (abs(gwySampX(i) - sensorToGwyTime(j)) < minVal)
```

```
48              minID = sensorToGwyTime(j);
49              minVal = abs(gwySampX(i) - sensorToGwyTime(j));
50              minDeltaT = gwySampX(i) - sensorToGwyTime(j);
51          end
52      end
53      evalTimes(i) = minID;
54      evalValues(i) = minVal;
55      evalDeltaT(i) = minDeltaT;
56  end
57
58  reconstructedPhasesBase = zeros(length(gwySampX), 1);
59  reconstructedPhasesFirst = zeros(length(gwySampX), 1);
60  reconstructedPhasesSecond = zeros(length(gwySampX), 1);
61  reconstructedPhasesThird = zeros(length(gwySampX), 1);
62  reconstructedPhasesFourth = zeros(length(gwySampX), 1);
63  reconstructedPhasesFifth = zeros(length(gwySampX), 1);
64
65  reconstructedCoefFirst = zeros(length(gwySampX), 1);
66  reconstructedCoefSecond = zeros(length(gwySampX), 1);
67  reconstructedCoefThird = zeros(length(gwySampX), 1);
68  reconstructedCoefFourth = zeros(length(gwySampX), 1);
69  reconstructedCoefFifth = zeros(length(gwySampX), 1);
70
71  for i = 1 : length(reconstructedPhasesBase)
72      for j = 1:length(sensorDataPhi)
73          if sensorToGwyTime(j) == evalTimes(i)
74              reconstructedPhasesBase(i) = sensorDataPhi(j) + evalDeltaT(
                    i) * signalFreq * 2 * pi;
75              reconstructedPhasesFirst(i) = sensorDataPhi(j) + evalDeltaT
                    (i) * signalFreq * 1 * 2 * pi;
76              reconstructedPhasesSecond(i) = sensorDataPhi(j) +
                    evalDeltaT(i) * signalFreq * 2 * 2 * pi;
77              reconstructedPhasesThird(i) = sensorDataPhi(j) + evalDeltaT
                    (i) * signalFreq * 3 * 2 * pi;
78              reconstructedPhasesFourth(i) = sensorDataPhi(j) +
                    evalDeltaT(i) * signalFreq * 4 * 2 * pi;
79              reconstructedPhasesFifth(i) = sensorDataPhi(j) + evalDeltaT
                    (i) * signalFreq * 5 * 2 * pi;
80
81              reconstructedCoefFirst(i) = sensorDataFirstHarmonic(j);
82              reconstructedCoefSecond(i) = sensorDataSecondHarmonic(j);
83              reconstructedCoefThird(i) = sensorDataThirdHarmonic(j);
84              reconstructedCoefFourth(i) = sensorDataFourthHarmonic(j);
```

```
            reconstructedCoefFifth(i) = sensorDataFifthHarmonic(j);
        end
    end
end


reconstructedBaseFncFirst = abs(reconstructedCoefFirst) .* exp(1j * (
   reconstructedPhasesFirst + angle(reconstructedCoefFirst)));
reconstructedBaseFncSecond = abs(reconstructedCoefSecond) .* exp(1j * (
   reconstructedPhasesSecond + angle(reconstructedCoefSecond)));
reconstructedBaseFncThird = abs(reconstructedCoefThird) .* exp(1j * (
   reconstructedPhasesThird + angle(reconstructedCoefThird)));
reconstructedBaseFncFourth = abs(reconstructedCoefFourth) .* exp(1j * (
   reconstructedPhasesFourth + angle(reconstructedCoefFourth)));
reconstructedBaseFncFifth = abs(reconstructedCoefFifth) .* exp(1j * (
   reconstructedPhasesFifth + angle(reconstructedCoefFifth)));

reconstructedSignal = 2 * real(reconstructedBaseFncFirst +
   reconstructedBaseFncSecond + ...
reconstructedBaseFncThird + reconstructedBaseFncFourth +
   reconstructedBaseFncFifth);

plot(reconstructedSignal)
hold on;
plot(gwySampY);
legend('Visszaállított␣jel', 'Gateway␣által␣mért␣jel');
grid on;
```

Az 2.6 mérés MATLAB-kódja

```
%% A 2.6 feladat: Fourier-egyutthatok becslese a gateway adataibol
clear;
clf;
clc;
close all;

%% Meresi adatok es parameterek betoltese
gwySampFull = load("Measurements/
   MasodikFeladatJel50HzKiBeKapcsoltProgi50Hz/pre_gwySampFull.dat");
sensorDataFull = load("Measurements/
   MasodikFeladatJel50HzKiBeKapcsoltProgi50Hz/pre_sensorResonator.dat")
   ;
stmpSync = load("Measurements/
   MasodikFeladatJel50HzKiBeKapcsoltProgi50Hz/pre_stmpSync.dat");
```

```
signalFreq = load("Measurements/
   MasodikFeladatJel50HzKiBeKapcsoltProgi50Hz/signalFreq.dat");

sensorDataID = sensorDataFull(:, 1);
sensorDataTime = sensorDataFull(:, 2);
sensorDataPhi = sensorDataFull(:, 3);
sensorDataFirstHarmonic = sensorDataFull(:, 4) + 1i * sensorDataFull(:,
    9);
sensorDataSecondHarmonic = sensorDataFull(:, 5) + 1i * sensorDataFull
   (:, 10);
sensorDataThirdHarmonic = sensorDataFull(:, 6) + 1i * sensorDataFull(:,
    11);
sensorDataFourthHarmonic = sensorDataFull(:, 7) + 1i * sensorDataFull
   (:, 12);
sensorDataFifthHarmonic = sensorDataFull(:, 8) + 1i * sensorDataFull(:,
    13);
clearvars sensorDataFull;

gwySampX = gwySampFull(:, 1);
gwySampY = gwySampFull(:, 2);
clearvars gwySampFull;

stmpSyncGwy = stmpSync(:, 1);
stmpSyncSens = stmpSync(:, 2);
clearvars stmpSync;

%% Gateway DC komponensenek eltavolitasa
gwySampY = gwySampY - mean(gwySampY);

%% Idobelyeg-transzformacio
idobelyegPoli = polyfit(stmpSyncSens, stmpSyncGwy, 1);
sensorToGwyTime = polyval(idobelyegPoli, sensorDataTime);

%% A szenzor fazisanak meghatarozasa a gateway mintavetelezesi
   idopontjaiban
% A gateway es a szenzor idoskala a szinkronizacios pontok alapjan
   azonos.
inSensorRange = gwySampX >= sensorToGwyTime(1) & gwySampX <=
   sensorToGwyTime(end);
validSamples = false(size(gwySampX));
validSamples(inSensorRange) = true;

nearestSensorIndex = nan(size(gwySampX));
```

```
45 nearestSensorIndex(inSensorRange) = interp1(sensorToGwyTime, (1:numel(
      sensorToGwyTime))', gwySampX(inSensorRange), "nearest");
46
47 gatewayPhase = nan(size(gwySampX));
48 for sampleIndex = find(inSensorRange)'
49     sensorIndex = nearestSensorIndex(sampleIndex);
50     gatewayPhase(sampleIndex) = sensorDataPhi(sensorIndex) + (gwySampX(
          sampleIndex) - sensorToGwyTime(sensorIndex)) * signalFreq * 2 *
          pi;
51 end
52
53 %% Fourier-egyutthatok becslese a rezonaros megfigyelovel
54 % A becsles a jegyzokonyv (8)-(10) egyenletei szerint tortenik:
55 % y'_n = 2 * real(c_n^T * xHat_n)
56 % e_n = y_n - y'_n
57 % xHat_(n+1) = xHat_n + alpha * e_n * conj(c_n)
58 harmonicNumbers = (1:5)';
59 alpha = 0.01;
60
61 reconstructedCoefFirst = nan(size(gwySampX));
62 reconstructedCoefSecond = nan(size(gwySampX));
63 reconstructedCoefThird = nan(size(gwySampX));
64 reconstructedCoefFourth = nan(size(gwySampX));
65 reconstructedCoefFifth = nan(size(gwySampX));
66 reconstructedSignal = nan(size(gwySampX));
67
68 estimatedCoefficients = zeros(numel(harmonicNumbers), 1);
69 validIndices = find(inSensorRange);
70 for sampleIndex = validIndices'
71     currentBasis = exp(1i * harmonicNumbers * gatewayPhase(sampleIndex)
          );
72     complexEstimate = currentBasis.' * estimatedCoefficients;
73     currentEstimate = 2 * real(complexEstimate);
74     currentError = gwySampY(sampleIndex) - currentEstimate;
75
76     reconstructedSignal(sampleIndex) = currentEstimate;
77     reconstructedCoefFirst(sampleIndex) = estimatedCoefficients(1);
78     reconstructedCoefSecond(sampleIndex) = estimatedCoefficients(2);
79     reconstructedCoefThird(sampleIndex) = estimatedCoefficients(3);
80     reconstructedCoefFourth(sampleIndex) = estimatedCoefficients(4);
81     reconstructedCoefFifth(sampleIndex) = estimatedCoefficients(5);
82
83     estimatedCoefficients = estimatedCoefficients ...
```

```
84          + alpha * currentError * conj(currentBasis);
85  end
86
87  %% A szenzor egyutthatoinak gateway idopontjaira illesztese
88  sensorCoefFirstAtGateway = nan(size(gwySampX));
89  sensorCoefSecondAtGateway = nan(size(gwySampX));
90  sensorCoefThirdAtGateway = nan(size(gwySampX));
91  sensorCoefFourthAtGateway = nan(size(gwySampX));
92  sensorCoefFifthAtGateway = nan(size(gwySampX));
93
94  for sampleIndex = find(inSensorRange)'
95      sensorIndex = nearestSensorIndex(sampleIndex);
96      sensorCoefFirstAtGateway(sampleIndex) = sensorDataFirstHarmonic(
            sensorIndex);
97      sensorCoefSecondAtGateway(sampleIndex) = sensorDataSecondHarmonic(
            sensorIndex);
98      sensorCoefThirdAtGateway(sampleIndex) = sensorDataThirdHarmonic(
            sensorIndex);
99      sensorCoefFourthAtGateway(sampleIndex) = sensorDataFourthHarmonic(
            sensorIndex);
100     sensorCoefFifthAtGateway(sampleIndex) = sensorDataFifthHarmonic(
            sensorIndex);
101 end
102
103 %% Hibak kiszamolasa
104 coefficientDifference = [
105     reconstructedCoefFirst(validSamples) - sensorCoefFirstAtGateway(
            validSamples), ...
106     reconstructedCoefSecond(validSamples) - sensorCoefSecondAtGateway(
            validSamples), ...
107     reconstructedCoefThird(validSamples) - sensorCoefThirdAtGateway(
            validSamples), ...
108     reconstructedCoefFourth(validSamples) - sensorCoefFourthAtGateway(
            validSamples), ...
109     reconstructedCoefFifth(validSamples) - sensorCoefFifthAtGateway(
            validSamples)];
110
111 coefficientRmsError = sqrt(mean(abs(coefficientDifference).^2, 1));
112 signalRmsError = sqrt(mean( ...
113     (gwySampY(validSamples) - reconstructedSignal(validSamples)).^2));
114
115 fprintf("Alfa = %.4f\n", alpha);
116 fprintf("Gateway es a visszaallitott jel RMS elterese = %.6g\n",
```

```
    signalRmsError);
117 fprintf("Fourier-egyutthatok RMS elterese:\n");
118 fprintf("  1. harmonikus: %.6g\n", coefficientRmsError(1));
119 fprintf("  2. harmonikus: %.6g\n", coefficientRmsError(2));
120 fprintf("  3. harmonikus: %.6g\n", coefficientRmsError(3));
121 fprintf("  4. harmonikus: %.6g\n", coefficientRmsError(4));
122 fprintf("  5. harmonikus: %.6g\n", coefficientRmsError(5));
123
124 %% A jel visszaallitasa es a gateway mereseinek osszehasonlitasa
125 figure;
126 plot(gwySampX(validSamples), gwySampY(validSamples));
127 hold on;
128 plot(gwySampX(validSamples), reconstructedSignal(validSamples));
129 xlabel("Ido [s]");
130 ylabel("Meresi ertek [ADC]");
131 title("A gateway jele es a Fourier-egyutthatokbol visszaallitott jel");
132 legend("Gateway altal mert jel", "Visszaallitott jel", ...
133     "Location", "best");
134 grid on;
135
136 %% A becsult es a szenzor altal szolgaltatott egyutthatok
    osszehasonlitasa
137 figure;
138 plot(gwySampX(validSamples), abs(reconstructedCoefFirst(validSamples)))
    ;
139 hold on;
140 plot(gwySampX(validSamples), abs(sensorCoefFirstAtGateway(validSamples)
    ));
141 plot(gwySampX(validSamples), abs(reconstructedCoefSecond(validSamples))
    );
142 plot(gwySampX(validSamples), abs(sensorCoefSecondAtGateway(validSamples
    )));
143 plot(gwySampX(validSamples), abs(reconstructedCoefThird(validSamples)))
    ;
144 plot(gwySampX(validSamples), abs(sensorCoefThirdAtGateway(validSamples)
    ));
145 plot(gwySampX(validSamples), abs(reconstructedCoefFourth(validSamples))
    );
146 plot(gwySampX(validSamples), abs(sensorCoefFourthAtGateway(validSamples
    )));
147 plot(gwySampX(validSamples), abs(reconstructedCoefFifth(validSamples)))
    ;
148 plot(gwySampX(validSamples), abs(sensorCoefFifthAtGateway(validSamples)
```

```
    ));
149 xlabel("Ido [s]");
150 ylabel("Fourier-egyutthato amplitudo");
151 title("A becsult es a szenzor altal szolgaltatott Fourier-egyutthatok")
    ;
152 legend("Becsult 1.", "Szenzor 1.", ...
153     "Becsult 2.", "Szenzor 2.", ...
154     "Becsult 3.", "Szenzor 3.", ...
155     "Becsult 4.", "Szenzor 4.", ...
156     "Becsult 5.", "Szenzor 5.", ...
157     "Location", "best");
158 grid on;
```